

AI Avatar Frontend Integration Guide

Prepared by: Franz Kingstein Nesakumar

Project: AI Avatar Interface

Frontend Stack: React 19 + Tailwind CSS + WebSocket

Note: This document is for backend and avatar engineers to understand how to plug their systems into the frontend we've completed.

What's Already Done (Frontend by Franz)

1. Frontend UI

- Responsive layout with:
 - Sidebar (placeholder links)
 - Avatar display area (`AvatarArea.jsx`)
 - Chat window with emoji picker and message flow (`ChatWindow.jsx`)
 - Control panel with buttons for:
 - Mic
 - Record
 - Volume
 - Repeat

2. WebSocket Integration

- `socket.io-client` is used and configured.
- Events:
 - Emit: `"user_message"` → Sends typed user message to backend.
 - Listen: `"bot_reply"` → Receives AI response text from backend and displays it in chat.

3. Component Wiring (Backend-Ready)

- Buttons in `ControlsPanel.jsx` are wired to emit events like:
 - `'record'`
 - `'mic'`
 - `'volume'`
 - `'repeat'`
 - You just need to listen for these events on your WebSocket backend.
- Avatar name is editable via an input field in `AvatarArea.jsx`.

Backend Developers — What You Need to Do

You **do not need to modify the frontend**. Just make your backend respond to the following expectations:

Frontend Action	Backend Should...
User sends message via chat	Listen to "user_message" and return AI reply via "bot_reply"
User clicks Mic / Repeat / Volume / Record	Listen to these events on the socket ('mic ', 'repeat ', etc.)
Avatar placeholder active	Send avatar-related animation or audio data to be rendered later (see below)

Backend Must Provide:

- A WebSocket server at:
`ws://localhost:8000` (or update the frontend path if needed)
- Emit `bot_reply` with just a plain string or JSON:

```
socket.emit("bot_reply", "This is the AI reply.");
```

The frontend already handles rendering this message.

Avatar Rendering — Integration Info

We've prepared the avatar display box (AvatarArea.jsx) as a placeholder.

You (or the 3D devs) can:

Insert a 3D canvas (like three.js, Babylon.js) in the `<div className="bg-gray-400">`

Add props like audio waveform or animation instructions

Hook into future socket events (e.g., avatar_lipsync) if needed

We Expect From Avatar Engineer:

The rendering logic for the avatar

A simple way to feed audio + lip-sync data (once we send it from backend)

Optionally, listen to bot_reply and animate lips accordingly

Summary

Area	Status	Notes
UI Layout	Complete	Responsive, interactive
Chat	Complete	WebSocket integrated
Controls	Event-ready	Emits events, backend just listens
WebSocket	Configured	Ready to send/receive
Avatar	Placeholder	Needs rendering logic from avatar engineer